

Trek Management Application

Modern Application Development I (MAD-I) Project Report

Rajnish Yadav

Roll No.: 25F2004238

25f2004238@ds.study.iitm.ac.in

1 Author

Name	Rajnish Yadav
Roll No.	25F2004238
Email	25f2004238@ds.study.iitm.ac.in
About Me	I am a student of the IIT Madras BS Degree in Data Science and Applications. This project was developed as part of the practical implementation for the course Modern Application Development I (MAD-I) , which focuses on web application development using the Flask framework.

1.1 AI / LLM Usage

AI Tool Used: ChatGPT

Extent of Usage

ChatGPT was primarily used for learning Flask concepts, debugging errors, improving the user interface, understanding reusable Jinja macros, refining the project documentation, and learning technologies used in the project such as Flask-WTF and WTFForms.

Approximately **15–20%** of the implementation consists of ideas or suggestions obtained while learning these concepts. For example, the implementation of Role-Based Access Control (RBAC) in the `decorators.py` module was inspired through learning with ChatGPT. All major application logic, database design, authentication, authorization, CRUD operations, and other core functionalities were designed and implemented independently.

Latex code generated through the ChatGPT from the self made docx file for report file to give simple professional touch to the pdf

Database Schema Tool Used: <https://dbdiagram.io/home>

Extent of Usage

DBDIAGRAMM is used for creating the **ERD image** (provide below) from the **database code written in models/model.py Only for Creating the Image not the Database Code**

1.2 Project Description

The Trek Management Application is developed using Python and the Flask web framework. It provides a role-based management system that allows administrators, trek staff, and trekkers to efficiently manage trekking activities, including user management, trek management, and booking management.

1.3 Technologies Used

Technology	Purpose
Python	Backend programming language used to implement the application logic.
Flask	Web framework used for application development.
Flask-SQLAlchemy	ORM used for interacting with the SQLite database.
Flask-Login	User authentication and session management.
Flask-WTF	Form creation and validation.
WTForms	HTML form generation and validation.
SQLite	Stores users, treks, bookings, and application data.
Jinja2	Template engine for rendering dynamic HTML pages.
HTML5	Web page structure.
CSS3	Styling and layout design.
Bootstrap 5	Responsive and modern user interface.
Git & GitHub	Version control and source code management.

Table 1: Technologies Used

2 Database Schema Design

The application uses a relational database designed using SQLAlchemy ORM with SQLite as the backend. The database consists of six tables: users, trekkers, staffs, admin, treks, bookings

2.1 ERD



Figure 1: ERD

2.2 Database Tables

Table	Primary Key	Foreign Key(s)	Purpose
users	id	—	Stores authentication details and common information for all users.
trekkers	user_id	user_id → users.id	Stores trekker-specific profile information and booking relationships.
staffs	user_id	user_id → users.id	Stores staff profile information and assigned treks.
admin	user_id	user_id → users.id	Stores administrator profile information.
treks	id	staff_id → staffs.user_id	Stores trek details including location, difficulty, duration, price, slots, and assigned staff.
bookings	id	trek_id → treks.id trekker_id → trekkers.user_id	Stores trek booking information including booking and payment status.

Table 2: Database Tables

2.3 Key Constraints

- Primary keys uniquely identify every record.
- Foreign keys maintain relationships between users, profiles, treks, and bookings.

- Email addresses in the Users table are unique.
- Trek codes are unique.
- Contact numbers are unique within profile tables.
- Mandatory attributes use NOT NULL constraints.
- Default values are defined for user roles, account status, booking status, and timestamps.

2.4 Reasons

The database has been normalized to reduce redundancy and improve maintainability.

- Common user information is stored only once in the **users** table.
- Role-specific information is stored separately in the **admin**, **staffs**, and **trekkers** tables using **one-to-one relationships**.
- Booking information is stored independently in the **bookings** table and connected to both trekkers and treks using **many-to-one relationships**.
- Trek information is stored in the **treks** table and linked to staff members, allowing one staff member to manage assigned multiple treks.

2.5 Relationship Summary

Parent Table	Child Table	Relationship
users	admin	1 : 1
users	staffs	1 : 1
users	trekkers	1 : 1
staffs	treks	1 : N
treks	bookings	1 : N
trekkers	bookings	1 : N

Table 3: Entity Relationships

3 API Design

No APIs used

4 Architecture and Features

4.1 Architecture

The project follows a modular Flask architecture that improves code readability, maintainability, and scalability.

The application starts from `app.py`, which creates the Flask application, initializes all required extensions, and registers the application blueprints.

Database models are defined using SQLAlchemy ORM inside the `models` package. User forms and validation are implemented using WTForms inside the `forms` package. Application controllers are organized into multiple Flask blueprints under the `routes` package for administrators, staff, trekkers, authentication, home pages, and error handling.

The presentation layer is implemented using Jinja2 templates and jinja macros with Bootstrap 5 and custom CSS (little bit of somewhere else bootstrap). Helper modules such as `filter.py` and `search.py` provide reusable searching, filtering, and sorting functionality. The `seed.py` module initializes the database and creates the default admin account.

4.2 Features Implemented

The application provides the following features:

- User registration and secure login.
- Role-Based Access Control (Admin, Staff, and Trekker).
- Admin approval for newly registered staff accounts.
- Admin management of users, staff accounts, and treks.
- Ability to approve, deactivate, or block users.
- Staff management of assigned treks.
- Staff management of booking status.
- Trekker booking management.
- Search, filtering, and sorting functionality.
- Role-specific dashboards.
- Separate management pages for treks, bookings, users, and booking history.

These features are implemented using Flask Blueprints, SQLAlchemy ORM, WTForms, Flask-Login, Jinja2, Bootstrap 5, and SQLite.

5 Video

Video Link : https://drive.google.com/file/d/18r53eKY_-cw0KvZnRJIgAUk98Ig_prSd/view?usp=sharing